

TEMA: Web Service REST

METODOLOGIA: Aprendizagem Baseada em Projetos

OBJETIVOS E HABILIDADES ENVOLVIDAS:

- Desenvolver um web service baseado na arquitetura REST.
- Persistência de dados usando *DAO* ou *Repository*.
- Ao menos um relacionamento de tabela N-N.
- Uso do formato JSON.
- Autenticação e validação de acesso.
- Uso de framework ORM.
- Uso dos códigos de mensagem HTTP.

ROTEIRO PARA ESTUDO:

API: <https://youtu.be/tPbK3eOJLXQ>

REST: <https://youtu.be/S7MduKwvVGk>

Especificações: <https://www.infoq.com/br/articles/rest-introduction/>

Sobre serviços e boas práticas: <https://www.alura.com.br/artigos/rest-principios-e-boas-praticas>

TRABALHO

ATENÇÃO: Ao lado de cada item está o peso da avaliação. Esse trabalho é dividido em 4 tópicos: **Persistência, REST, Segurança e Autenticação**

Com base no material estudado sobre JPA e Spring, desenvolva um Web Service usando a arquitetura REST seguindo os seguintes critérios:

Escolha um tema junto com sua equipe. Os temas são livres.

Persistência

1. A aplicação deve possuir conexão com o banco de dados relacional (MySQL, MariaDB, SQL Server, PostgreSQL). **(3,0)**

2. Deve haver pelo menos 2 tabelas que se relacionam, de modo que o relacionamento entre elas seja N-N, gerando assim uma terceira tabela de normalização de dados. **(3,0)**
3. O projeto deve ser desenvolvido utilizando o padrão *DAO* ou *Repository*, e a arquitetura MVC. Use uma framework ORM para auxiliar. **(4,0)**

REST:

1. Para cada classe de domínio, deve-se fornecer os seguintes serviços em formato JSON:
 - a. insert **(2,0)**
 - b. delete **(2,0)**
 - c. update **(2,0)**
 - d. select by id **(1,0)**
 - e. select all **(1,0)**

Cada serviço acima deve ser implementado utilizando framework ORM. No *controller*, realizar validação de dados, lançando exceções conforme os possíveis erros previstos:

- a. Registro não encontrado.
- b. Algum erro ao acessar o banco de dados.
- c. Falta de algum parâmetro obrigatório para consulta ou persistência.
- d. Erro de integridade, que pode ocorrer ao deletar um registro usado em outro cadastro.

Lançar exceção com os códigos de erro HTTP **(2,0)**

SEGURANÇA:

1. Além dos objetos já definidos, deve-se implementar serviços para gerenciar o usuário do sistema. Implementar o **CRUD** (*Create, Read, Update, Delete*) com nome, login e senha. **(3,0)**
2. A senha do usuário deve ser criptografada. **(2,0)**

3. Os serviços da aplicação devem ser acessados somente com autenticação.
Nesse caso, o sistema deve permitir fazer login e cadastrar um usuário novo sem autenticação, e os demais serviços (*endpoints*) devem validar autenticação.
(5,0)

AUTENTICAÇÃO:

1. A autenticação se dá pelo login e senha. A camada de autenticação deve gerar um token, pode ser usado JWT (*JSON Web Tokens*) para que o cliente possa acessar os outros *endpoints* sem precisar enviar o usuário e senha novamente.
(6,0)
2. A sessão deve ficar ativa por 5 minutos. (2,0)
3. Na apresentação do trabalho, preparar todas as chamadas dos *endpoints* que foram desenvolvidos pela equipe em um ambiente de testes, como o Postman, Insomnia, cURL. (2,0)

ENTREGÁVEIS

- Criar o repositório no GitHub para o trabalho e adicionar o usuário **gabrielvra** como colaborador. O professor vai acompanhar todo o desenvolvimento por lá. É importante que todos os membros da equipe realizem *commits* no repositório.
- Todos os membros da equipe devem apresentar o trabalho para o professor.
- Após finalizar, fazer um zip do projeto com o código fonte e anexar no AVA.